

Aplikacja webowa R Shiny – porównanie metod regresji, klasyfikacji i klasteryzacji

Raport projektowy

Autorzy: Dawid Balikowski, Piotr Graczyk

Repozytorium: github.com/balikowski/DBPG-LAB

Spis treści

1	Wstęp	2
2	Opis zbiorów danych	2
2.1	Iris	2
2.2	Boston	2
2.3	mtcars	2
2.4	Wczytany plik CSV	3
3	Algorytmy i moduły	3
3.1	Wstępne przetwarzanie danych	3
3.2	Metody regresji	3
3.2.1	Regresja liniowa (OLS)	3
3.2.2	Random Forest dla regresji	3
3.2.3	Support Vector Regression (SVR)	4
3.3	Metody klasyfikacji	4
3.3.1	Regresja logistyczna	4
3.3.2	Support Vector Machine (SVM)	4
3.3.3	Random Forest dla klasyfikacji	4
3.4	Metody klasteryzacji	5
3.4.1	K-means	5
3.4.2	DBSCAN	5
3.4.3	Klasteryzacja hierarchiczna (hclust)	5
4	Opis aplikacji	5
4.1	Struktura plików	5
4.2	Interfejs użytkownika	6
5	Wyniki badań	7
5.1	Regresja	7
5.2	Klasyfikacja	8
5.3	Klasteryzacja	10
6	Wnioski	11

1. Wstęp

Projekt stanowi interaktywna aplikacja webowa zbudowana w technologii R Shiny, której celem jest umożliwienie eksploracji i porównania popularnych metod uczenia maszynowego w trzech kategoriach: regresji, klasyfikacji i klasteryzacji. Aplikacja pozwala użytkownikowi na:

- wczytanie własnego zbioru danych w formacie CSV lub skorzystanie z wbudowanych zbiorów,
- interaktywny dobór zmiennych wejściowych i docelowych,
- konfigurację parametrów modeli (podział trening/test, skalowanie, balansowanie klas, liczba klastrów, metoda łączenia),
- wizualizację wyników za pomocą zestawów wykresów diagnostycznych,
- obliczanie predykcji dla nowych, ręcznie wprowadzonych obserwacji.

Kod źródłowy projektu dostępny jest publicznie pod adresem <https://github.com/balikowski/DBPG-LAB>. Poniżej opisano budowę aplikacji, zastosowane algorytmy, dane oraz uzyskane wyniki.

2. Opis zbiorów danych

Aplikacja udostępnia trzy wbudowane zbiory danych i obsługuje import pliku CSV. Każdy zbiór jest automatycznie wstępnie przetwarzany przez funkcję `preprocess_data()` opisaną w sekcji 3.1.

2.1. Iris

Zbiór *Iris*, dostępny bezpośrednio w bazowym środowisku R. Zawiera 150 obserwacji i 5 kolumn: cztery cechy kwiatów (długość i szerokość działki kielicha *Sepal.Length*, *Sepal.Width* oraz płatk *Petal.Length*, *Petal.Width*, wyrażone w centymetrach) i jedną kolumnę kategoryjną (*Species*: *setosa*, *versicolor*, *virginica*). Zbiór jest zbilansowany – każda klasa liczy dokładnie 50 obserwacji. W projekcie był głównie wykorzystywany do testów klasyfikacji wieloklasowej i klasteryzacji.

2.2. Boston

Zbiór *Boston Housing*, dostępny w pakiecie *MASS*. Zawiera 506 obserwacji i 14 zmiennych opisujących dzielnicę Bostonu (m.in. wskaźnik przestępczości *crim*, proporcja gruntów nierezydencjalnych *indus*, stężenie tlenków azotu *nox*, średnia liczba pokoi *rm*, odległość od centrum zatrudnienia *dis*). Zmienna docelowa *medv* oznacza medianę wartości domów właścicielsko-zajętych w tysiącach dolarów. Zbiór był stosowany w zadaniach regresji.

2.3. mtcars

Zbiór *Motor Trend Car Road Tests*, dostępny bezpośrednio w R. Zawiera 32 obserwacje (modele samochodów) oraz 11 zmiennych numerycznych, m.in.: *mpg* (zużycie paliwa, mile na galon), *hp* (moc silnika), *wt* (masa pojazdu), *cyl* (liczba cylindrów). Zbiór był stosowany w regresji (*mpg* jako zmienna docelowa) oraz klasyfikacji binarnej (np. *am*: skrzynia manualna/automatyczna).

2.4. Wczytany plik CSV

Aplikacja umożliwia import dowolnego pliku CSV (limit: 100 000 wierszy, co najmniej 2 kolumny). Użytkownik może wybrać separator (., ; lub tabulacja) oraz określić, czy plik posiada wiersz nagłówkowy. Po wczytaniu dane są automatycznie przetwarzane.

3. Algorytmy i moduły

3.1. Wstępne przetwarzanie danych

Moduł `preprocess.R` definiuje funkcję `preprocess_data()`, uruchamianą automatycznie dla każdego wczytywanego zbioru. Funkcja wykonuje następujące kroki:

1. Zamiana pustych ciągów znaków ("") na wartości brakujące (NA).
2. Usunięcie wierszy i kolumn złożonych wyłącznie z NA.
3. Ujednolicenie nazw kolumn funkcją `make.names()` (unikanie konfliktów ze składnią R).
4. Usunięcie zduplikowanych wierszy.
5. Próba konwersji kolumn tekstowych na numeryczne – jeśli ponad 80% wartości daje się sparsować jako liczba, kolumna jest konwertowana.
6. Uzupełnienie braków: dla kolumn numerycznych – medianą; dla kolumn tekstowych – etykietą "Brak".

Dodatkowa funkcja `safe_encode_x()` (moduł `model_spec.R`) obsługuje enkodowanie zmiennych kategorycznych przed dopasowaniem modeli klasyfikacji. Kolumny z co najwyżej 20 unikalnymi wartościami poddawane są *label encodingowi* lub *one-hot encodingowi* (do wyboru przez użytkownika; domyślnie one-hot). Kolumny o wyższej liczbie unikalnych wartości zastępowane są zestawem 5 cech tekstowych (długość ciągu, liczba słów, cyfr, wielkich liter, znaków specjalnych).

3.2. Metody regresji

Wszystkie trzy metody regresji używają domyślnego podziału **80% trening / 20% test** (`set.seed(123)`), bez możliwości zmiany tego proporcji przez użytkownika. Metryki obliczane są na zbiorze testowym.

3.2.1 Regresja liniowa (OLS)

Klasyczny model liniowy dopasowywany funkcją `lm()` z pakietu bazowego R. Model przyjmuje formułę $Y \sim X_1 + X_2 + \dots + X_p$ i szacuje współczynniki metodą najmniejszych kwadratów. Raportowane metryki (na zbiorze testowym): R^2 , R^2_{adj} , MSE, RMSE. Wykresy diagnostyczne: obserwowane vs. przewidywane, diagram reszt, wykres Q-Q, macierz korelacji cech, przedziały ufności współczynników (95% CI) i histogram rozkładu reszt z nałożoną krzywą normalną.

3.2.2 Random Forest dla regresji

Model lasów losowych (`randomForest::randomForest()`) z domyślnymi parametrami (`ntree=500`; reprodukowalność: `set.seed(123)`). Predykcje i metryki obliczane są na 20-procentowym zbiorze testowym, co zapewnia rzetelną ocenę zdolności generalizacji modelu. Raportowane metryki: R^2 , R^2_{adj} , MSE, RMSE oraz ważność zmiennych (`IncNodePurity`).

Wykresy: analogiczne do regresji liniowej, uzupełnione wykresem ważności zmiennych i krzywą błędu OOB w funkcji liczby drzew.

3.2.3 Support Vector Regression (SVR)

Model regresji wektorami nośnymi (`e1071::svm()`) z jądrem radialnym (RBF): `cost=1`, `epsilon=0.1`. Dane liczbowe są standaryzowane przed dopasowaniem z użyciem parametrów wyznaczonych na zbiorze treningowym (średnia i odchylenie standardowe), a następnie identyczne parametry stosowane są do skalowania zbioru testowego. Parametr `gamma` wyznaczany automatycznie na podstawie wariancji cech. Raportowane metryki (na zbiorze testowym): R^2 , R^2_{adj} , MSE, RMSE, MAE. Wykresy: m.in. wizualizacja ϵ -tube z zaznaczonymi support vectors i histogram rozkładu $|reszt|$ z podziałem na obserwacje w/poza ϵ -tube.

3.3. Metody klasyfikacji

Każdy model klasyfikacji korzysta z podziału trening/test konfigurowalnego suwakiem (zakres 50–90%, krok 5%, wartość domyślna: 80%). Dostępne opcje dodatkowe:

- **Skalowanie** danych liczbowych: wyłączone (domyślnie), standaryzacja lub normalizacja.
- **Enkodowanie** zmiennych kategorycznych: one-hot encoding (domyślnie) lub label encoding.
- **Balansowanie klas**: brak (domyślnie) lub automatyczne wagi klas (`class_weights`).

3.3.1 Regresja logistyczna

Dla problemów binarnych stosowana jest funkcja `glm(family=binomial())`; dla problemów wieloklasowych – `nnet::multinom()`. Próg decyzyjny dla problemu binarnego wynosi 0,5. Raportowane metryki: dokładność (accuracy), macierz pomyłek, precision, recall i F1 per klasa. Dla klasyfikacji binarnej generowana jest krzywa ROC z wartością AUC. Wykresy diagnostyczne obejmują m.in. rozkład predykowanych prawdopodobieństw z zaznaczoną granicą 0,5 (binarnie) lub licznik poprawnych/błędnych predykcji (wieloklasowo) oraz wykres bezwzględnych wartości współczynników.

3.3.2 Support Vector Machine (SVM)

Model SVM z jądrem radialnym (`e1071::svm()`, `kernel="radial"`, `cost=1`, `probability=TRUE`, `scale=FALSE`). Skalowanie obsługiwane jest ręcznie przez aplikację zgodnie z ustawieniem wybranym przez użytkownika. Metryki: analogiczne do regresji logistycznej. Wykresy: scatter plot dwóch pierwszych cech numerycznych z zaznaczonymi poprawnie i błędnie sklasyfikowanymi obserwacjami oraz wykres liczby support vectors per klasa.

3.3.3 Random Forest dla klasyfikacji

Las losowy (`randomForest()`, `ntree=500`, `importance=TRUE`) z miarą ważności zmiennych `MeanDecreaseGini`. Metryki: accuracy, wskaźnik błędu OOB, macierz pomyłek, precision, recall, F1. Wykresy: krzywa OOB error w funkcji liczby drzew, ważność zmiennych i macierz korelacji cech numerycznych.

3.4. Metody klasteryzacji

3.4.1 K-means

Algorytm k-średnich (`kmeans()`) ze standaryzacją danych (`scale()`), `nstart=25`, `iter.max=100` i ziarnem losowości `set.seed(123)`. Użytkownik wybiera liczbę klastrow k suwakiem (zakres 2–10, domyślnie 3). Raportowane wskaźniki: WCSS, BSS, TSS, stosunek BSS/TSS oraz średnia szerokość sylwetki (pakiet `cluster`). Wykresy: scatter plot z centroidami, metoda łokcia, wykres sylwetki, liczebność klastrow, heatmapa centroidów w przestrzeni znormalizowanej i średnia sylwetka per klastery.

3.4.2 DBSCAN

Algorytm gęstościowy DBSCAN (`dbscan::dbscan()`) z automatycznym doбором parametru ε jako 90. percentyl rozkładu odległości do (`minPts` – 1)-ego sąsiada (k-NN distance plot). Parametr `minPts` jest konfigurowalny przez użytkownika suwakiem (zakres 2–20, krok 1, wartość domyślna: 5). Wyniki: liczba wykrytych klastrow, liczba punktów szumu (noise), sylwetka (z wykluczeniem szumu). Wykresy: scatter z zaznaczonym szumem, k-NN distance plot z linią ε , liczebność klastrow, wykres sylwetki, histogram odległości od centroidu klastra i wykres kołowy udziału szumu w zbiorze.

3.4.3 Klasteryzacja hierarchiczna (`hclust`)

Algorytm klasteryzacji hierarchicznej aglomeracyjnej (`hclust()` z pakietu bazowego R) z danymi standaryzowanymi (`scale()`) i macierzą odległości euklidesowych (`dist()`). Użytkownik wybiera:

- **Liczba klastrow** k – suwakiem (zakres 2–10, domyślnie 3),
- **Metodę łączenia** – `ward.D2` (domyślnie), `complete`, `average` lub `single`.

Podział na klastry realizowany jest przez `cutree(hc, k)`. Raportowane: metoda łączenia, liczba klastrow, liczebność każdego klastra oraz średnia szerokość sylwetki. Wykresy (6 paneli w układzie 2×3):

1. Scatter plot dwóch pierwszych cech z centroidami (gwiazdki),
2. Dendrogram (próbka maks. 60 obserwacji) z czerwoną linią cięcia dla wybranego k ,
3. Liczebność klastrow (słupkowy),
4. Wykres sylwetki,
5. Heatmapa centroidów klastrow w przestrzeni standaryzowanej,
6. Boxplot pierwszej wybranej cechy per klastery.

4. Opis aplikacji

4.1. Struktura plików

DBPG-LAB-main/

```
app.R           # punkt wejścia (shinyApp)
server.R        # logika serwera
ui.R            # interfejs użytkownika (navbarPage)
R/
  model_spec.R  # specyfikacje i implementacje modeli
```

```

preprocess.R    # czyszczenie i uzupełnianie danych
server_helpers.R # helpery: update/show/setup inputs
ui_components.R # wielokrotne komponenty UI

```

4.2. Interfejs użytkownika

Aplikacja wykorzystuje układ `navbarPage` z pięcioma głównymi zakładkami:

- **Dane** – import CSV i podgląd wybranego zbioru (10 pierwszych wierszy + struktura).
- **Regresja** – zakładki dla regresji liniowej, RF i SVR.
- **Klasyfikacja** – zakładki dla regresji logistycznej, SVM i RF.
- **Klasteryzacja** – zakładki dla K-means, DBSCAN i klasteryzacji hierarchicznej (`hclust`).
- **Informacje o aplikacji** – opis projektu, twórcy i link do repozytorium.

Każda metoda regresji i klasyfikacji posiada trzy podzakładki: *Model* (konfiguracja i podsumowanie tekstowe), *Wykresy* (sześć wykresów diagnostycznych w układzie 2×3) oraz *Predykcja* (dynamiczne inputy generowane na podstawie wybranych kolumn, wynik po kliknięciu przycisku). Metody klasteryzacji wyświetlają konfigurację, wyniki tekstowe i wykresy w jednym widoku.

The screenshot shows the application's 'Dane' (Data) tab. At the top, there's a navigation bar with tabs: DBPG-LAB, Dane (active), Regresja, Klasyfikacja, Klasteryzacja, and Informacje o aplikacji. Below the navigation bar, there are two buttons: 'Import' and 'Podgląd' (Preview). The main content area is titled 'Podgląd danych' (Data Preview). It includes a dropdown menu labeled 'Wybierz dane' (Select data) with 'iris' selected. Below this is a table showing the first 10 rows of the 'iris' dataset. The table has columns: Sepal.Length, Sepal.Width, Petal.Length, Petal.Width, and Species. The data shows various measurements for the 'setosa' species. Below the table, there's a section titled 'Informacje o danych' (Data Information) which displays summary statistics: 'Liczba wierszy: 149' (Number of rows: 149), 'Liczba kolumn: 5' (Number of columns: 5), and the column names: 'Sepal.Length', 'Sepal.Width', 'Petal.Length', 'Petal.Width', and 'Species'. It also shows the data structure: 'data.frame' with 149 observations and 5 variables, and a list of variables with their data types and levels.

Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
5.10	3.50	1.40	0.20	setosa
4.90	3.00	1.40	0.20	setosa
4.70	3.20	1.30	0.20	setosa
4.60	3.10	1.50	0.20	setosa
5.00	3.60	1.40	0.20	setosa
5.40	3.90	1.70	0.40	setosa
4.60	3.40	1.40	0.30	setosa
5.00	3.40	1.50	0.20	setosa
4.40	2.90	1.40	0.20	setosa
4.90	3.10	1.50	0.10	setosa

Informacje o danych

Liczba wierszy: 149
Liczba kolumn: 5

Nazwy kolumn:
[1] "Sepal.Length" "Sepal.Width" "Petal.Length" "Petal.Width" "Species"

Struktura danych:
'data.frame': 149 obs. of 5 variables:
 \$ Sepal.Length: num 5.1 4.9 4.7 4.6 5.4 4.6 5 4.4 4.9 ...
 \$ Sepal.Width: num 3.5 3 3.2 3.1 3.6 3.9 3.4 3.4 2.9 3.1 ...
 \$ Petal.Length: num 1.4 1.4 1.3 1.5 1.4 1.7 1.4 1.5 1.4 1.5 ...
 \$ Petal.Width: num 0.2 0.2 0.2 0.2 0.2 0.4 0.3 0.2 0.2 0.1 ...
 \$ Species : Factor w/ 3 levels "setosa","versicolor",...: 1 1 1 1 1 1 1 1 1 1 ...

Rysunek 1: Widok ogólny aplikacji – zakładka „Dane / Podgląd”

Rysunek 2: Zakładka „Regresja / Regresja liniowa” – konfiguracja modelu i podsumowanie

5. Wyniki badań

Poniżej przedstawiono wybrane wyniki uzyskane dla poszczególnych kategorii metod. Eksperymenty przeprowadzono na trzech wbudowanych zbiorach danych, z różnymi kombinacjami parametrów.

5.1. Regresja

Do porównania metod regresji wykorzystano zbiory *Boston* (zmienna docelowa: `medv`) i *mtcars* (zmienna docelowa: `mpg`) z pełnym zestawem zmiennych numerycznych. Wszystkie modele trenowane były na 80% obserwacji, metryki obliczane na pozostałych 20% (zbiór testowy).

Tabela 1: Porównanie metod regresji – zbiór *Boston* (zmienna docelowa: `medv`, podział 80/20)

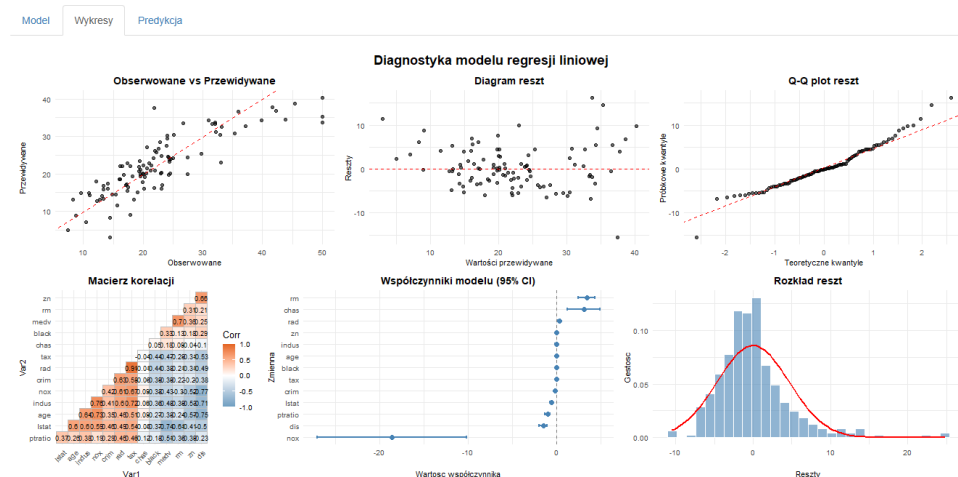
Metoda	R^2	R^2_{adj}	MSE	RMSE
Regresja liniowa	0.719	0.678	23.679	4.866
Random Forest	0.865	0.845	11.382	3.374
SVR (RBF)	0.854	0.832	12.340	3.513

Tabela 2: Porównanie metod regresji – zbiór *mtcars* (zmienna docelowa: *mpg*, podział 80/20)

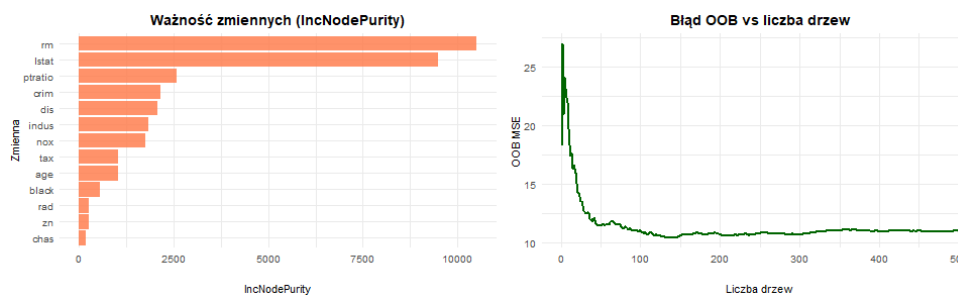
Metoda	R^2	MSE	RMSE
Regresja liniowa	0.434	6.707	2.590
Random Forest	0.588	4.878	2.209
SVR (RBF)	0.342	7.795	2.792

Uwaga: Kolumna R^2_{adj} została pominięta, ponieważ przy 7 obserwacjach testowych i 10 predyktorach mianownik wzoru $n_{test} - p - 1 = -4$ jest ujemny, przez co wskaźnik traci interpretację statystyczną. Miarodajną miarą dopasowania pozostaje R^2 .

Regresja liniowa



Rysunek 3: Wykresy diagnostyczne regresji liniowej – zbiór Boston (układ 2x3)



Rysunek 4: Wykresy diagnostyczne Random Forest – zbiór Boston (ważność zmiennych, OOB)

5.2. Klasyfikacja

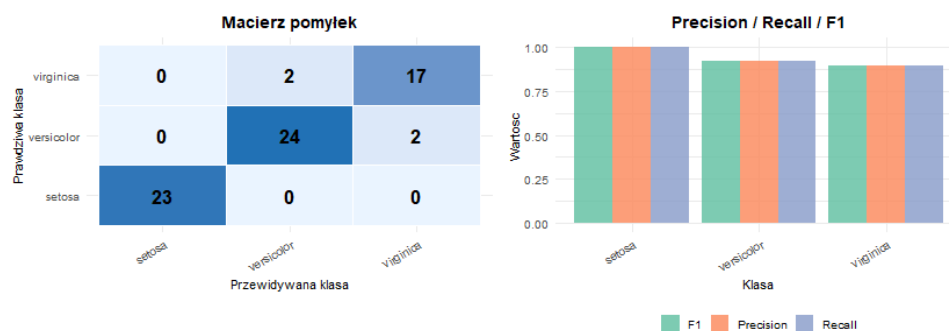
Klasyfikację testowano na zbiorach *Iris* (3 klasy: *Species*) i *mtcars* (2 klasy: *am* – rodzaj skrzyni biegów). Dla wszystkich modeli zastosowano podział 55/45%. Poniższe tabele przedstawiają wyniki dla zbioru *Iris* oraz dla *mtcars*.

Tabela 3: Porównanie metod klasyfikacji – zbiór *Iris* (Species, podział 55/45)

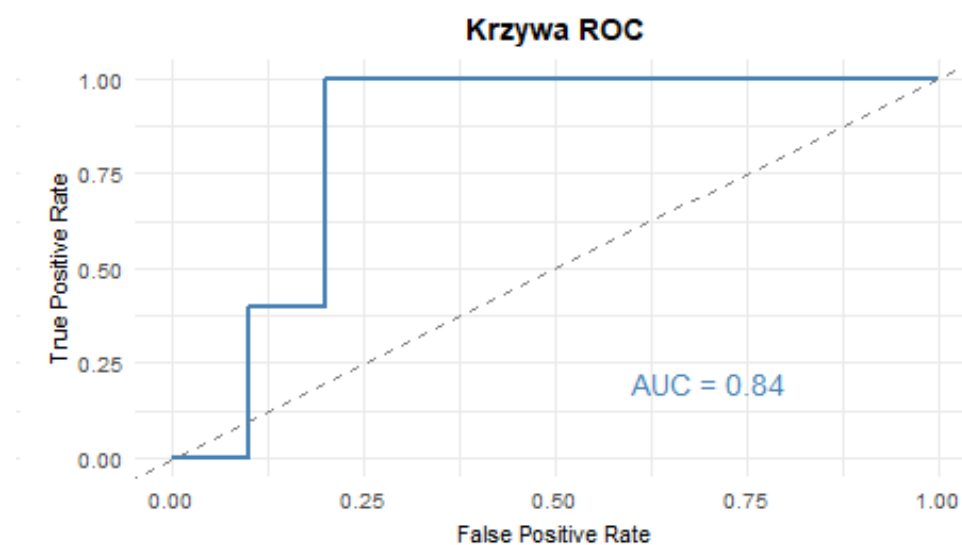
Metoda	Accuracy	Precision (avg)	Recall (avg)	F1 (avg)
Regresja logistyczna	0.9412	0.9406	0.9470	0.9415
SVM (RBF, cost=1)	0.9559	0.9533	0.9568	0.9548
Random Forest	0.9412	0.9393	0.9393	0.9393

Tabela 4: Wpływ skalowania na accuracy – SVM, zbiór *mtcars* (zmienna *am*, podział 55/45)

Skalowanie	Accuracy
Brak (none)	0.375
Standaryzacja	1.000
Normalizacja	0.875



Rysunek 5: Macierz pomyłek i wykresy Precision/Recall/F1 – Random Forest, zbiór Iris

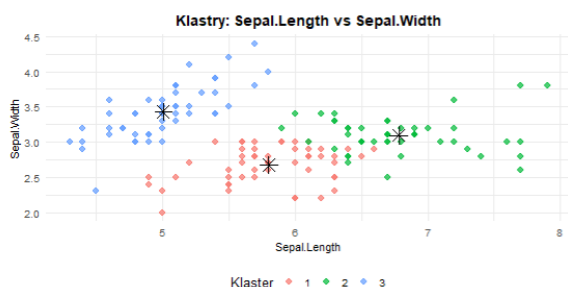
Rysunek 6: Krzywa ROC (AUC = 0.84) – Regresja logistyczna, zbiór *mtcars* (klasyfikacja binarna *am*)

5.3. Klasteryzacja

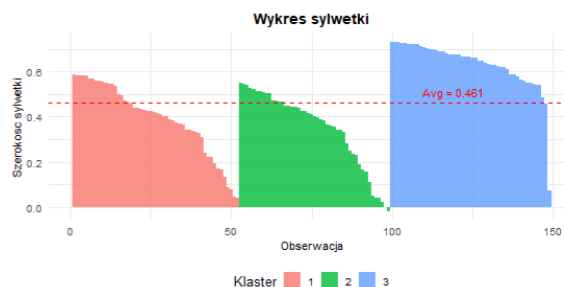
Klasteryzację przeprowadzono na zbiorze *Iris* z użyciem cech numerycznych (*Sepal.Length*, *Sepal.Width*, *Petal.Length*, *Petal.Width*).

Tabela 5: Porównanie metod klasteryzacji – zbiór *Iris* (4 cechy numeryczne)

Metoda	Liczba klastrów	Śr. sylwetka	BSS/TSS
K-means ($k = 2$)	2	0.5810	0.6292
K-means ($k = 3$)	3	0.4607	0.7672
K-means ($k = 4$)	4	0.3875	0.8102
DBSCAN (minPts=5, auto- ϵ)	2	0.5995	–
hclust (ward.D2, $k = 3$)	3	0.4498	–
hclust (ward.D2, $k = 2$)	2	0.5763	–



(a) Scatter plot z centroidami

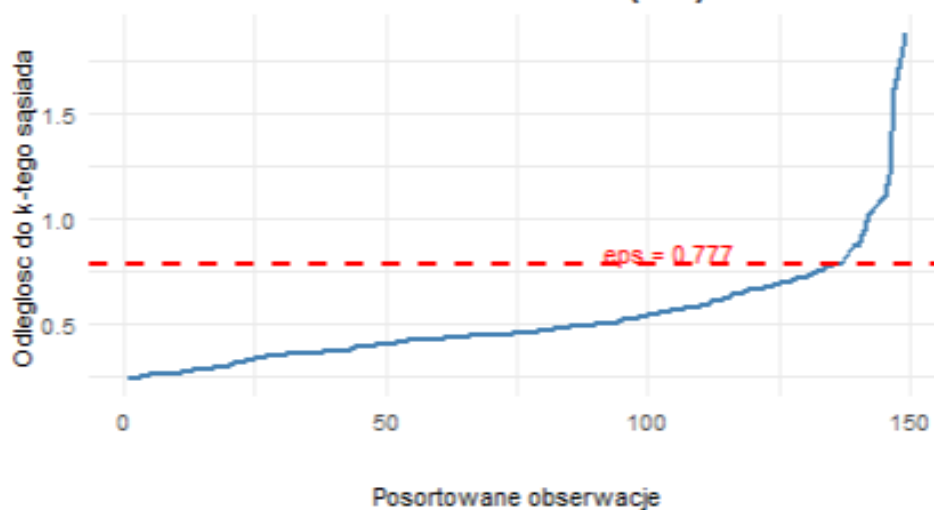


(b) Wykres sylwetki

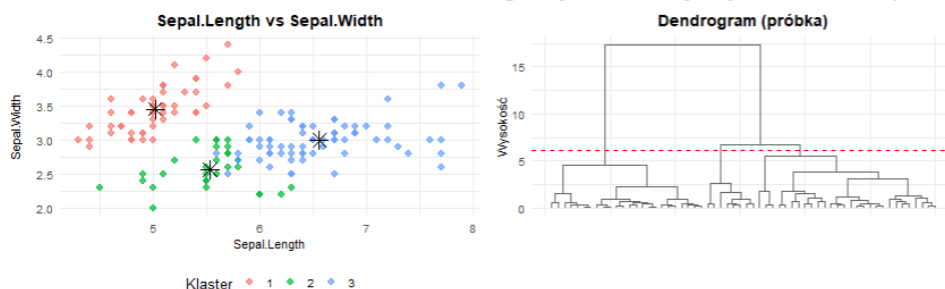
Rysunek 7: K-means ($k = 3$) – scatter plot z centroidami i wykres sylwetki, zbiór *Iris*

Diagnostyka – DBSCAN

k-NN Distance Plot ($k=4$)



Rysunek 8: DBSCAN – k-NN distance plot ($\epsilon=0.777$, minPts=5, $k=4$) i wizualizacja klastrów, zbiór *Iris*



Rysunek 9: Klasteryzacja hierarchiczna (ward.D2, $k = 3$) – dendrogram z linią cięcia i scatter plot, zbiór Iris

6. Wnioski

Aplikacja realizuje kompletne środowisko do interaktywnej analizy danych z perspektywy uczenia maszynowego. Zaimplementowane moduły umożliwiają użytkownikowi samodzielne eksplorowanie wpływu parametrów modeli na jakość wyników. Z obserwacji nad działaniem aplikacji wynikają następujące spostrzeżenia:

- **Regresja:** Na zbiorze *Boston* (506 obserwacji) Random Forest (RMSE = 3.374) i SVR (RMSE = 3.513) wyraźnie poprawiają wyniki względem regresji liniowej (RMSE = 4.866), co potwierdza zdolność obu modeli do uchwycenia nieliniowych zależności. Na zbiorze *mtcars* (zaledwie 7 obserwacji testowych) Random Forest nadal wypada najlepiej (RMSE = 2.209), natomiast SVR przy ustalonych hiperparametrach i tak małym zbiorze treningowym uzyskuje gorsze wyniki niż regresja liniowa (RMSE = 2.792 vs 2.590). Wskaźnik R^2_{adj} dla *mtcars* jest statystycznie nieinterpretowany z powodu ujemnego mianownika wzoru ($n_{test} - p - 1 = -4$) i został pominięty w tabeli.
- **Klasyfikacja:** Na zbiorze *Iris* ze zbiorem testowym stanowiącym 45% danych SVM (accuracy = 0.9559) uzyskał nieznacznie lepsze wyniki od regresji logistycznej i Random Forest (oba = 0.9412). Skalowanie ma fundamentalny wpływ na skuteczność SVM – dla zbioru *mtcars* (zmienna *am*, podział 55/45) brak skalowania skutkuje dokładnością zaledwie 0.375, podczas gdy standaryzacja podnosi ją do wartości doskonałej (1.000), a normalizacja daje 0.875. Wynik ten jednoznacznie potwierdza wrażliwość SVM na skalę cech wejściowych i uzasadnia domyślną standaryzację stosowaną w bibliotece *e1071*.
- **Klasteryzacja:** Najwyższą średnią sylwetkę spośród testowanych metod osiągnął DBSCAN (0.5995), łącząc nakładające się gatunki *versicolor* i *virginica* w jeden zwarty klaster. K-means z $k=3$ (sylwetka = 0.461, BSS/TSS = 0.767) daje strukturę zbliżoną do rzeczywistego trójgatunkowego podziału zbioru. Klasteryzacja hierarchiczna metodą Ward’a uzyskuje sylwetki porównywalne z K-means przy tym samym k (hclust $k=3$: 0.450 vs K-means $k=3$: 0.461) i dostarcza dendrogramu ułatwiającego intuicyjny dobór liczby klastrów.
- **Przetwarzanie danych:** Automatyczne uzupełnianie braków medianą i etykietą "Brak" oraz enkodowanie zmiennych kategorycznych (`safe_encode_x()`) znacząco upraszczają pracę z różnorodnymi danymi wejściowymi bez ingerencji użytkownika.